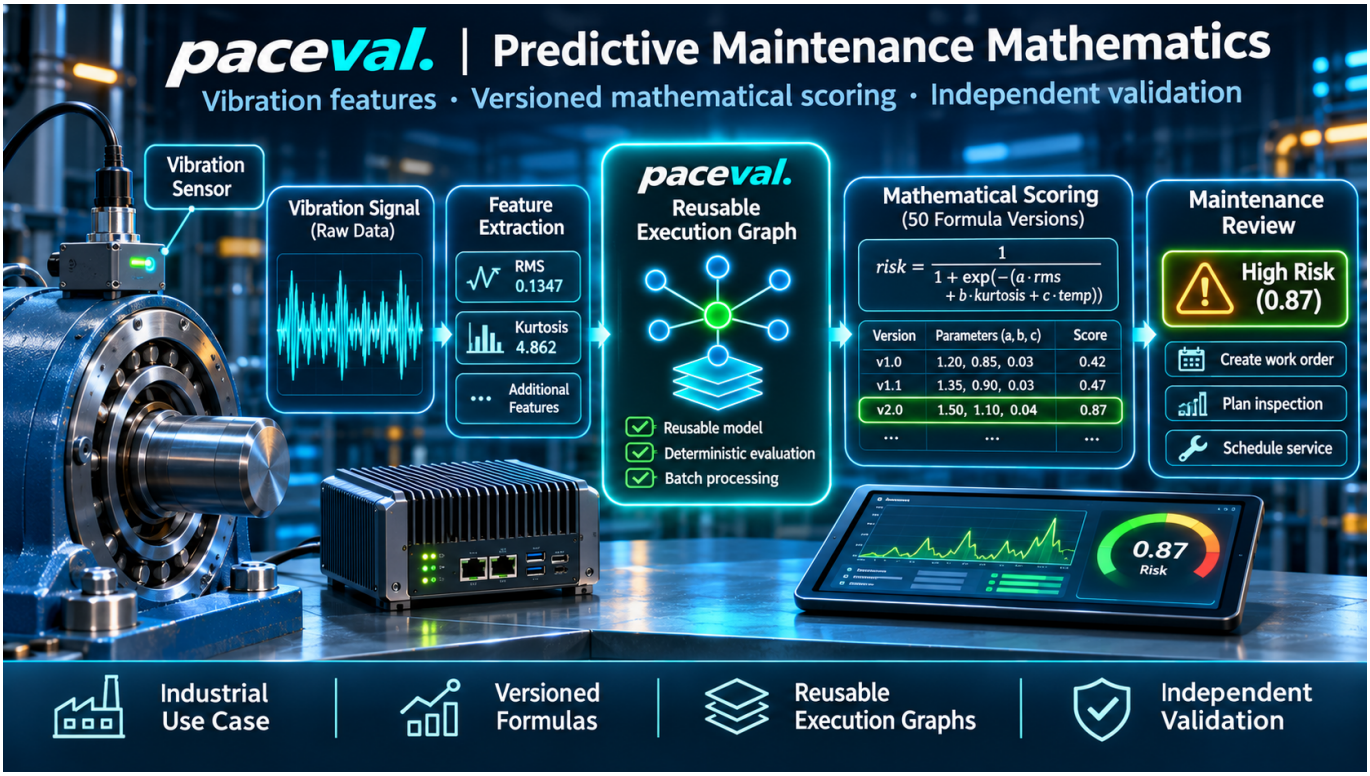
Mathematical Predictive-Maintenance Scores with paceval

Reproducible vibration-feature scoring with 50 reusable mathematical graphs



Illustrative application visual. The hardware scene is not a photograph of a tested deployment.

Executive research result

The research proposal WP-006 has been implemented as an executable C++17 study using the original Linux paceval runtime. Vibration features are computed from labelled synthetic acceleration windows; 50 prepared mathematical score expressions are then evaluated repeatedly through the official C/C++ SDK. A frozen model is selected using asset-disjoint training and validation partitions, followed by a separate held-out test.

Scientific scope	Observed result
Synthetic study cohort	48 assets × 12 windows = 576 windows; 512 acceleration samples per window
Reusable paceval formula graphs	50 regular scoring graphs plus one separate TINC graph
Numerical agreement	28,800/28,800 accepted scores pass both native and 90-digit independent checks
Sample-level TINC audit	24/24 interval samples include the high-precision reference
Frozen held-out decision	TP 71, TN 72, FP 0, FN 1 (synthetic data only)

Research date: 11 November 2026. This report describes executed synthetic feasibility tests, not an operational industrial fault-prediction product.

1. Scientific motivation and precise evaluation boundary

Predictive maintenance frequently uses physical measurements such as vibration, current, operating state and temperature. A common research design extracts stable features from time-series windows and evaluates a fixed, inspectable scoring rule. A prepared mathematical-expression runtime is appropriate for the latter step. It is not, by itself, a fault-label source, vibration sensor, supervised classifier, prognostics model or maintenance decision authority.

WP-006 proposed the logistic function risk = 1/(1+exp(-(a·rms + b·kurtosis + c·temp))) and fifty replaceable score variants. Here it is operationalised as a dimensionless score with declared feature normalization, four formula families and 50 fixed version identifiers. The synthetic coefficients are not trained to estimate event probabilities.

Open recordings for a subsequent external pilot

- **Paderborn University KAt Bearing DataCenter.** Offers synchronized vibration and electric-current signals, operating conditions and temperature; published data are CC BY-NC 4.0. This report neither redistributes nor evaluates those recordings [1].
- **CWRU Bearing Data Center.** Documents healthy and faulted bearing acceleration traces under specified operating loads. Temperature is not a guaranteed companion channel for the formula in this study [2].
- **NASA IMS bearing repository.** Provides bearing-degradation recordings for future evaluation of a genuinely time-evolving condition index [3].

The study therefore avoids presenting artificial labels as externally observed bearing failures. For a real-data experiment, splitting by bearing/asset rather than overlapping windows is essential to avoid leakage. Validated data provenance and aligned temperature measurements would be required.

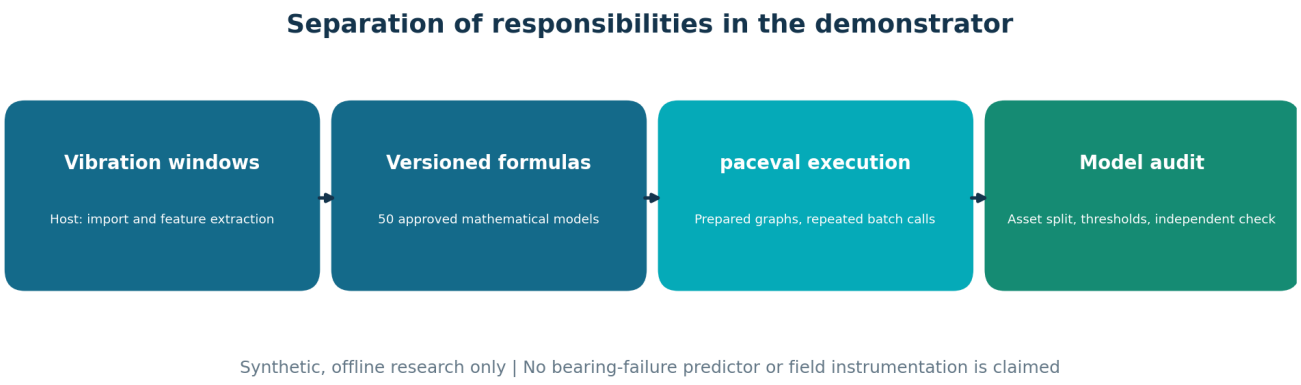


Figure 1. The host handles acquisition, features and decisions; paceval evaluates versioned mathematical expressions.

2. Mathematical model and reproducible formula families

Vibration features. For raw samples x[j] in a window of n samples, the C++ host evaluates the root mean square and Pearson population kurtosis:

> RMS = sqrt((1/n) sum x[j]^2) | mu = (1/n) sum x[j]
>
> Kurtosis = ((1/n) sum (x[j] - mu)^4) / (((1/n) sum (x[j] - mu)^2)^2)

This definition does not subtract 3. A zero-variance window is rejected because kurtosis is undefined. Acceleration is expressed in synthetic g units and the temperature input is synthetic degrees Celsius.

Normalization and mathematical scoring. The host passes R = RMS/0.3, K = (Kurtosis - 3)/3 and T = (temperature_C - 60)/20 to paceval. The normalized arguments and coefficients are dimensionless. Every formula computes:

> S(R,K,T) = 1/(1 + exp(-z)), where z = aR + bK + cT + d + correction_family(R,K).

Family	Additional explicit mathematical term
0	No additional term
1	0.035 × R × K
2	0.03 × R × R
3	0.045 × tanh(K)

Each of the fifty versioned models uses one of these families and a documented, distinct coefficient tuple, defined in **src/predictive_maintenance.cpp**. These are fixed mathematical research examples, not fifty independently trained prognostic systems. A fixed decision threshold S ≥ 0.55 is used for the study; no threshold tuning is performed on the test set.

3. C++ / paceval implementation

- Compile **src/paceval_main.cpp** alongside **src/predictive_maintenance.cpp** and include **src/paceval_main.h**; link the externally supplied original Linux `.so`.
- Create each computation with **paceval_CreateComputation** and evaluate repeated sample-major batches through **paceval_dGetComputationResultExt**.
- Check creation errors and per-sample evaluation codes; release all graphs with **paceval_DeleteComputation** and close the runtime with **paceval_FreeLibrary**.
- Evaluate a separate TINC-enabled graph on 24 selected samples with **paceval_dGetComputationResult**.

4. Study design, synthetic cohort and partitioning

A deterministic Python generator (seed 20261111) produces 48 labelled synthetic assets, 12 measurement windows per asset and 512 acceleration values per window. The signals combine periodic vibration, bounded noise and added impulses for the positive synthetic label. Temperature is generated as part of the toy model; no measurement uncertainty or laboratory calibration is asserted.

Research partition	Assets	Feature windows	Purpose
Training	24	288	Rank the 50 preset formula versions
Validation	12	144	Select from a shortlist of 10 pre-ranked formulas
Held-out test	12	144	Report frozen-model classification only

The same asset does not appear in different partitions. Input-order constraints and data validation are enforced in the C++ host. Importers must provide real labels and a documented sensor-temperature channel; missing values cannot be silently fabricated.

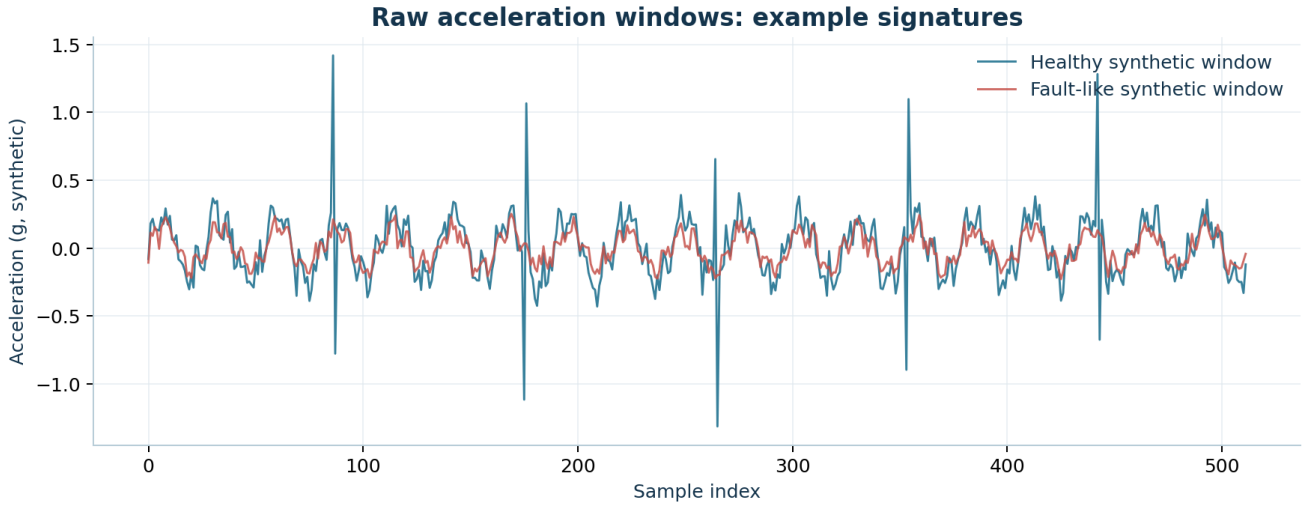


Figure 2. Illustrative healthy and fault-like vibration windows from the generated synthetic cohort.

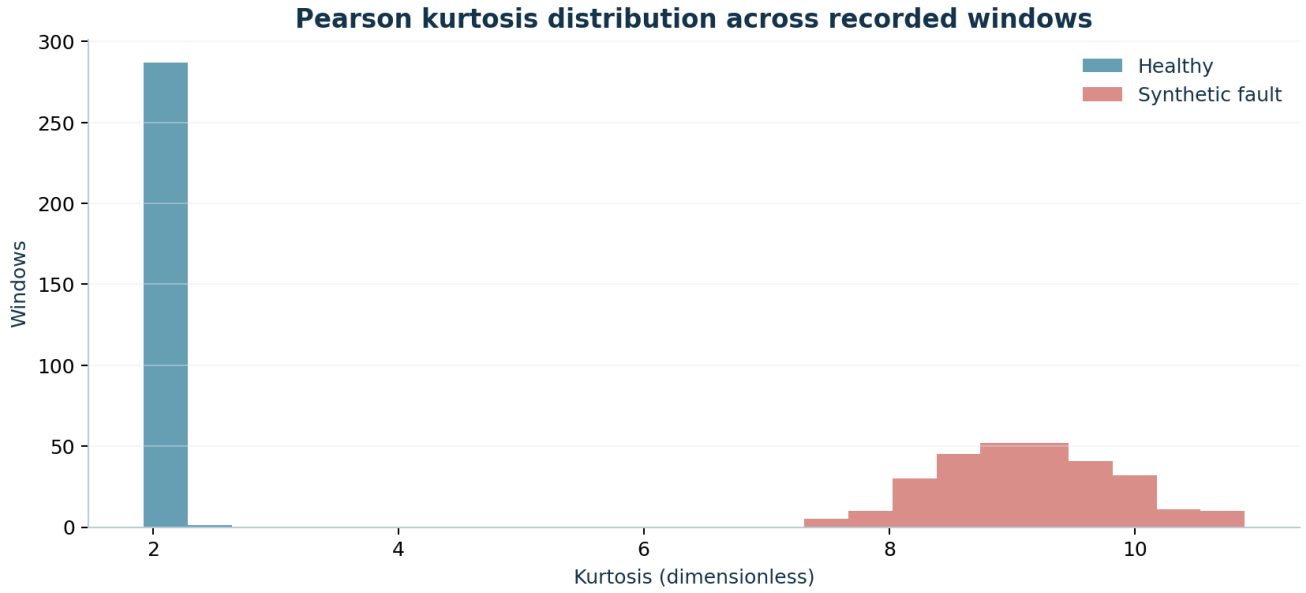


Figure 3. Distribution of Pearson kurtosis calculated from the complete synthetic cohort.

5. Model selection and previously unseen synthetic assets

All 50 formula versions are evaluated using prepared paceval graphs. The research host computes fixed-threshold F1 scores on training assets, retains ten models, and chooses the highest-validation-F1 candidate. The held-out test partition is then reported for the frozen choice. In this deterministic study model 5 (v1.5) is selected.

Hold-out statistic	Measured value
True positives	71
True negatives	72
False positives	0
False negatives	1
Precision	1.0000
Recall	0.9861
F1	0.9930
Balanced accuracy	0.9931

These favourable numbers describe artificial signatures designed to differ between synthetic labels. They cannot be extrapolated to unseen equipment, realistic class prevalence or true maintenance outcomes.

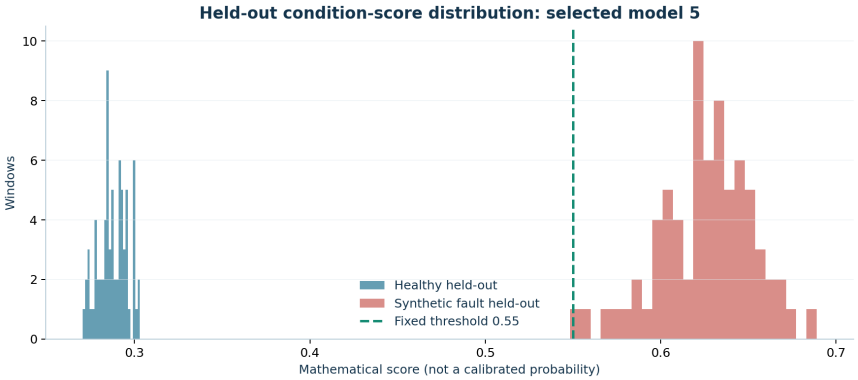


Figure 4. Distribution of mathematical scores among held-out synthetic windows; fixed decision threshold 0.55.

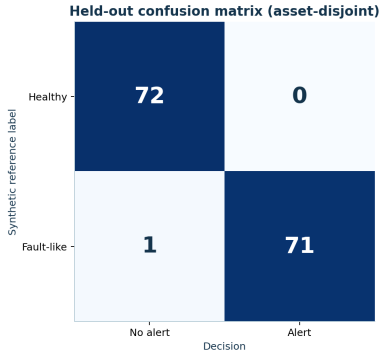


Figure 5. Confusion matrix from the held-out synthetic assets. No physical bearing failures were predicted.

6. Multiple formula versions and interpretable response surfaces

The mathematical contribution of paceval is that fifty distinct, versioned score expressions can be prepared and reused with new feature values. The engine remains responsible for repeatable mathematical evaluation; the study host maintains model metadata, the shortlist, thresholds and statistical reporting.

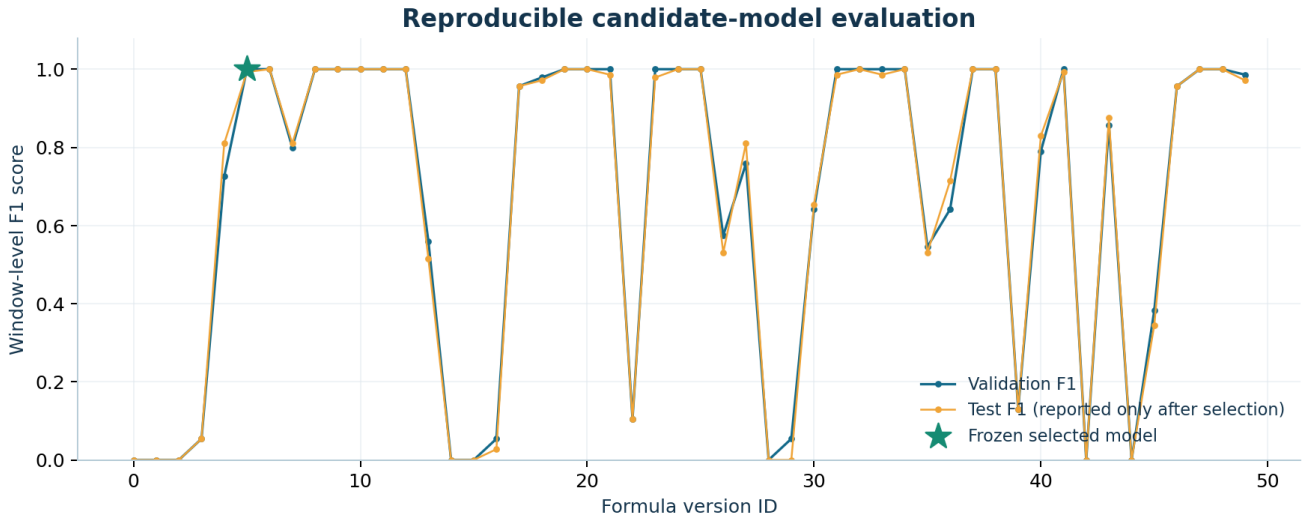


Figure 6. Frozen per-version performance. Test metrics shown for analysis were not used to choose the winner.

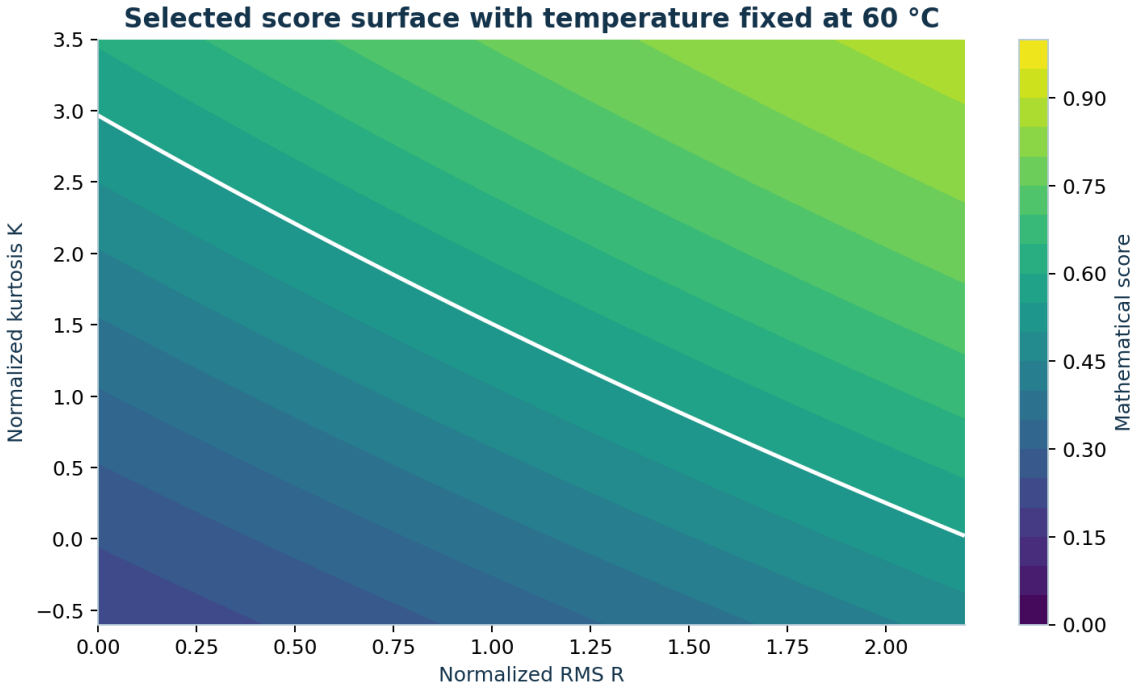


Figure 7. Response surface of the selected formula, holding temperature at 60 °C. It is an illustrative, dimensionless mathematical score.

7. Independent numerical validation and TINC samples

All 28,800 published paceval scores were compared against an independently written native C++ mathematical reference during execution. A second, separate Python implementation using mpmath with 90 decimal digits re-evaluated all 28,800 exported results. Both checks recorded zero tolerance violations (numerical criterion: absolute difference at most 3e-11 for the native reference, and 3e-11 × (1+|reference|) for the mpmath audit).

The independently computed maximum absolute difference across the exported results is approximately **2.35×10^{-16}**. Errors can depend on the formula and input conditioning, so this maximum is descriptive of this finite sample only.

A separate paceval graph created with the TINC option provided numerical lower and upper values for 24 preselected inputs. All 24 intervals contain their independently recomputed 90-digit sample reference. This is a sample-level observation, not a universal proof of outward-rounded interval arithmetic.

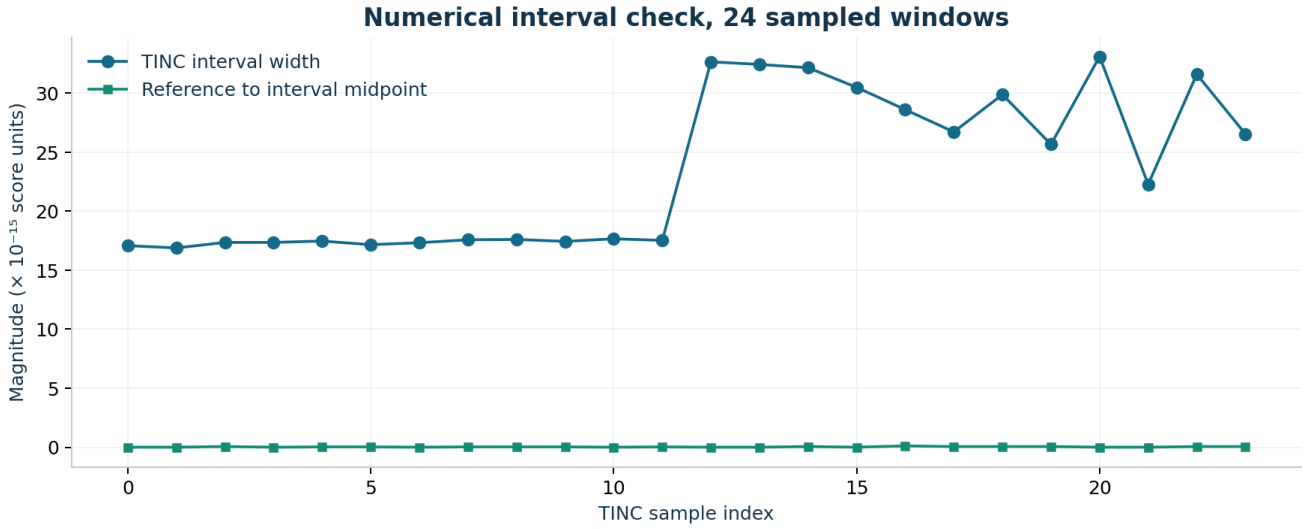


Figure 8. Numeric widths of the sampled intervals and offsets of their independent references.

Reproduction and software-quality checks

- A native feature/mathematical self-test verifies RMS, kurtosis, sigmoid scores and invalid inputs.
- The original Linux runtime is loaded directly through the vendor-provided SDK bridge and exercised in full-batch tests.
- Two dataset-invalidity checks reject malformed fields and inconsistent asset partition metadata (see the C++ host and examples).
- Published CSV files include per-version scores, features, models and independent audit results; the complete program can be re-run from a clean CMake build.

8. Integration boundary, reproducibility and next research stage

A future industrial pipeline would receive time-stamped acceleration readings from validated sensors; compute stable vibration features; pass normalized, finite numerical inputs into approved paceval expressions; and return a condition index for downstream review. Time-series acquisition, feature windows, rule versioning, failure labels, alert persistence and maintenance actions remain responsibilities of the application and organization.

The strongest next experiment is a documented, bearing-disjoint study using licensed data with confirmed fault metadata, correlated operating conditions, and a temperature field. Results should include precision, recall, false alarms per operating hour, false negatives, threshold robustness and calibration across independent equipment and loads. The current raw-data CSV schema supports externally prepared recordings; external datasets themselves are not bundled.

Available publication files

Artifact	Relative GitHub path
Code and CMake	src/predictive_maintenance.cpp; src/CMakeLists.txt
paceval SDK sources	src/paceval_main.h; src/paceval_main.cpp
Synthetic data generator	src/generate_data.py
Independent high-precision audit	src/high_precision_reference.py
Reproducible scientific figures	src/make_figures.py; figures/
Synthetic data / output	examples/raw_vibration.csv; examples/evaluation.csv
Model-level measurements	examples/model_metrics.csv; examples/tinc_samples.csv
Research summary / build instructions	README.md; paceval_Predictive_Maintenance_Whitepaper_EN.pdf

Source references

[1] Paderborn University KAt Bearing DataCenter, operating conditions and measured vibration/current/temperature data. https://mb.uni-paderborn.de/en/kat/research/bearing-datacenter/data-sets-and-download

[2] Case Western Reserve University Bearing Data Center, documented healthy and seeded-defect acceleration recordings. https://engineering.case.edu/bearingdatacenter/download-data-file

[3] NASA Prognostics Center of Excellence, IMS bearing recordings. https://data.nasa.gov/dataset/ims-bearings

[4] paceval, official C/C++ API and prepared computation integration. https://paceval.com/api/

[5] Original research proposal WP_006_Mathematical_Predictive_Maintenance_Scores.pdf, supplied as context. Not required to build the GitHub demonstrator.